

Tema 4: Cifrado asimétrico

Docente : Héctor Xavier Limón Riaño
email: xavier120@hotmail.com

Introducción

Introducción

- ▶ El cifrado asimétrico es uno de los avances más importantes en seguridad criptográfica
- ▶ Está detrás de la seguridad de la web, conexiones Wi-Fi, intercambio de correo, entre muchas otras cosas
- ▶ Pero...tiene muchas sutilidades y es fácil de usar e implementar de forma incorrecta

- ▶ En realidad, las técnicas criptográficas asimétricas son variadas, no se limitan sólo a cifrado
- ▶ Por ejemplo: firmas digitales e intercambio de llaves
- ▶ Es este tema nos centraremos en cifrado utilizando el algoritmo RSA
- ▶ Existen otros algoritmos importantes modernos, especialmente aquellos derivados de curvas elípticas (se verán en temas posteriores)

Introducción

- ▶ RSA está de hecho casi obsoleto, debido a que se han descubierto algunas vulnerabilidades y a que es ineficiente
- ▶ Sin embargo, sirve para ejemplificar cifrado asimétrico

Dos llaves

- ▶ El principio del cifrado asimétrico es que se necesitan dos llaves: una llave pública y una llave privada
- ▶ Si un mensaje se cifra con la llave pública, sólo el poseedor de la llave privada puede descifrar el mensaje y vice-versa
- ▶ La llave pública se comparte, mientras que la privada nunca debe compartirse

Dos llaves

- ▶ El tener dos llaves bajo este esquema mejora sustancialmente la seguridad
- ▶ Por ejemplo en cifrado simétrico: la llave puede comprometerse del lado del receptor o del emisor, comprometiendo todos los mensajes
- ▶ Imagina en una guerra donde capturan al espía que tiene una llave simétrica: será posible comprometer todos los mensajes
- ▶ En el mismo escenario con llaves asimétricas: sólo la llave pública estaría comprometida, a los captores no les sería posible descifrar lo que el espía ha enviado, pues no tienen la llave privada (aunque si podrían descifrar los mensajes que le han llegado)

RSA

- ▶ No se presentará a profundidad el algoritmo
- ▶ Las llaves son básicamente dos números enteros primos muy grandes
- ▶ Estos números son coprimos, osea que el máximo común divisor que tienen es 1
- ▶ Dichos números son además los factores primos de un número aun más grande que sólo tiene 2 factores primos
- ▶ Todos los números (excepto los primos) se pueden descomponer en factores primos, osea en multiplicaciones de números primos
- ▶ Por ejemplo: $10 = 5 * 2$, $24 = 2 * 2 * 2 * 3$

RSA

- ▶ Si solo se tiene uno de los dos números primos (osea una de las llaves), para un numero grande (2048 bits por ejemplo), es computacionalmente intratable encontrar el número para el cual esta llave es uno de los factores primos (y por lo tanto cual es el otro factor primo)
- ▶ RSA utiliza también exponenciación modular
- ▶ RSA es relativamente fácil de implementar en lenguajes de programación modernos

RSA

- ▶ Encontrar dos números grandes co-primos es una tarea compleja que requiere pruebas estadísticas, en este caso se simplifica con el siguiente código:
- ▶ Ver archivo compartido atajos/RSA/llaves.py
- ▶ La llave pública se deriva a partir de la privada, pero no se puede al revés (también por eso son asimétricas)
- ▶ PEM es un formato estándar muy utilizado de codificación, para almacenar llaves
- ▶ Para simplificar, no se cifró en este caso la llave privada, aunque debe hacerse

RSA

- ▶ A partir de la generación de llaves vista, se implementará manualmente las operaciones de cifrado y descifrado
- ▶ Dicha implementación es insegura y **NUNCA** debe usarse, se presenta con fines didácticos para entender como funciona RSA y que problemas de seguridad tiene
- ▶ Más adelante se verán dichas funciones con la biblioteca adecuada
- ▶ Para dicha implementación se necesita la biblioteca gmpy2 que cuenta con una operación de potencia modular

RSA

- ▶ Para cifrar:
 - ▶ $c \equiv m^e \pmod{n}$
- ▶ Para descifrar:
 - ▶ $m \equiv c^d \pmod{n}$
- ▶ Donde:
 - ▶ c es un número entero que se corresponde al mensaje cifrado
 - ▶ m es un número entero que se corresponde al mensaje plano
 - ▶ e , d y n son parámetros que se pueden extraer a partir de las llaves

RSA

- ▶ Ver atajos/RSA/rsaManual.py
- ▶ Dado que RSA trabaja con enteros y no bytes, la implementación por defecto puede perder ceros
- ▶ Por ejemplo: 01 es lo mismo que 1 en números enteros, pero no en bytes
- ▶ El código mostrado no lidia con este problema, si el archivo a ser procesado empieza con un 0, entonces se descartarán los 0, lo cual puede dar un resultado incorrecto
- ▶ Para que funcione bien, se requiere agregar padding (otra vez...)

Ejercicios

- ▶ Crear un script que genera la llave pública y privada en las rutas elegidas por el usuario (a partir de dos parámetros) y guarda el resultado en formato PEM
- ▶ Crear un script que recibe como parámetro la ruta de una llave pública o privada en formato PEM, la ruta de un archivo de texto plano o cifrado (por ahora sólo archivos de texto) y la operación a realizar (cifrar o descifrar)

Llave pública

- ▶ La llave pública como su nombre lo dice, puede ser pública, en la mayoría de los casos, no importa si un atacante la tiene
- ▶ Sólo la persona con la llave privada podrá descifrar los mensajes que se cifren con la llave pública
- ▶ Ahora bien, al poseedor de la llave privada le resultara difícil saber si puede confiar en la información que le envían, pero ese es un problema de autenticación que se verá en otro tema

mitm

- ▶ Notar que por si mismo el cifrado asimétrico no resuelve todos los problemas de seguridad, he incluso introduce nuevos
- ▶ Por ejemplo: imaginar un ataque mitm donde el atacante intercepta la llave pública de una entidad A, dicha llave está destinada a una entidad B
- ▶ El atacante crea su propias llaves y le da su llave pública a B, haciéndole pensar que es de A
- ▶ Mientras tanto, el atacante cifra mensajes arbitrarios con la llave pública de A, para hacerle pensar que a A que B se comunica con él
- ▶ B estará generando mensajes privados cifrados mediante la llave pública del atacante, los cuales podrán ser descifrados por el atacante

Llave privada

- ▶ El poseedor de una llave privada protegida apropiadamente puede usar cifrado asimétrico para dos cosas:
 - ▶ Buzón de mensajes criptográficos: cualquiera con la llave pública puede crear y enviar un mensaje para el propietario de la llave privada
 - ▶ Firmas: cualquiera con la llave pública puede descifrar mensajes creados con la llave privada. Esto obviamente no ayuda a la confidencialidad, pero si ayuda a probar la identidad del emisor ya que para haber creado el mensaje debe poseer la llave privada

Limitaciones RSA

- ▶ RSA casi nunca se utiliza para servir de buzón de mensajes criptográficos ya que está pensado para mensajes cortos
- ▶ De hecho su mayor uso tenía (tenía porque está siendo remplazado por curvas elípticas) que ver con cifrar llaves simétricas para intercambio en protocolos criptográficos como TLS
- ▶ En realidad el cifrado asimétrico es en general mucho más ineficiente que el simétrico, los protocolos criptográficos modernos usan cifrado simétrico para el cifrado de los datos
- ▶ El cifrado asimétrico es más utilizado para establecer identidad (lo cual se verá en el siguiente tema del curso) e intercambiar llaves simétricas (también se verá en un tema posterior)

Cifrado asimétrico

- ▶ Una de las mayores ventajas es que ni el emisor ni el receptor tienen que ponerse de acuerdo para empezar a transmitir mensajes, a diferencia del cifrado simétrico donde las dos partes tienen que intercambiar la llave
- ▶ Aunque esta ventaja puede también ser una desventaja, como ya se mencionó en la sección de mitm
- ▶ Este tipo de problemas se resuelve con certificados (los cuales se verán en el siguiente tema)

Ataques

Padding RSA

- ▶ No ponerle padding a RSA es un error bastante grave
- ▶ Es tan grave que la biblioteca cryptography ni siquiera deja cifrar con RSA sin padding
- ▶ Pero, ¿Qué es el padding y porqué es tan importante?
- ▶ La mejor forma de demostrar su importancia es viendo cómo pueden leerse mensajes cifrados con la llave pública, sin conocer la llave privada

Padding RSA

- ▶ RSA es un algoritmo determinista, esto es, dado la misma llave y mensaje produce siempre la misma salida cifrada
- ▶ Este problema ya se vio con AES (se resolvía con el IV)
- ▶ El padding en este caso ayuda a tener algo similar al IV
- ▶ El problema del determinismo es que le permite al atacante encontrar patrones
- ▶ Por ejemplo, si el atacante sabe que m se cifra a c , cada vez que vea c sabrá cual es el texto plano

Padding RSA

- ▶ El atacante puede tener la llave pública y conocer el algoritmo de cifrado (nunca se debe asumir que el atacante no conoce el cifrado utilizado)
- ▶ El atacante puede cifrar muchos mensajes potenciales, y guardarlos en una tabla lookup para hacer mapeos entre mensajes planos y su correspondiente cifrado
- ▶ Especialmente si los mensajes son cortos este tipo de técnica funciona

Padding RSA

- ▶ Incluso si el atacante no puede adivinar el contenido del mensaje, puede analizar los patrones en mensajes repetidos
- ▶ Esto le puede ayudar al atacante a suponer lo que dicen los mensajes o a plantear otro tipo de explotaciones (por ejemplo repetir el mismo mensaje y hacerse pasar por la víctima)

Ataque de texto cifrado escogido

- ▶ RSA sin padding es vulnerable a este ataque
- ▶ Este ataque consiste en hacer que la víctima descifre algunos mensajes escogidos por el atacante para el atacante, de esta forma el atacante puede descifrar cualquiera de los otros mensajes (se verá demostración más adelante)
- ▶ ¿Porqué la víctima descifraría mensajes para el atacante?
- ▶ La seguridad tiene muchos aspectos psicológicos humanos, todo esto se abarca en ingeniería social

Ataque de texto cifrado escogido

- ▶ Imagina el siguiente escenario: la víctima recibe un mensaje supuestamente de alguien de confianza, al descifrarlo encuentra texto sin sentido
- ▶ ¿La víctima asume que es un ataque?
- ▶ Puede ser que si, pero también puede ser que asuma que hubo un error en la transmisión
- ▶ Si la víctima tiene pocos controles de seguridad, podría simplemente desechar el mensaje y dejarlo en un lugar inseguro (un bote de basura por ejemplo)

Ataque de texto cifrado escogido

- ▶ ¿Qué es más fácil para el atacante, hacerse con mensajes que viajan por un canal con seguridad reforzada o hacerse con mensajes que son desechados en la basura?
- ▶ Si el atacante tiene a alguien infiltrado, encontrar los mensajes desechados tiene una posibilidad más alta
- ▶ Obviamente esta es una forma simplista de ver el problema, un ataque de este tipo es más complejo

Ataque de texto cifrado escogido

- ▶ Si el atacante puede recuperar los mensajes desechados descifrados (incorrectos para la víctima) entonces va a poder descifrar cualquier otro mensaje cifrado con la misma llave pública
- ▶ Para lograrlo es necesario aplicar algunos principios matemáticos, especialmente el de **homomorfismo** en cifrado

Ataque de texto cifrado escogido

- ▶ La idea básica del homomorfismo en cifrado es que si se realiza un cambio computacional en el texto cifrado, el resultado se ve reflejado en el texto plano descifrado
- ▶ No todos los sistemas criptográficos tienen homomorfismo, pero RSA lo tiene hasta cierto punto
- ▶ En RSA hay formas de hacer una multiplicación en el texto cifrado que resulta en multiplicaciones en el texto plano descifrado

Ataque de texto cifrado escogido

- ▶ El homomorfismo es un todo un tema en criptografía, incluso existen y se están desarrollando servicios que manipulan datos cifrados sin necesidad de poder leerlos
- ▶ Esta propiedad limitada de homomorfismo de RSA abre las puertas a varias vulnerabilidades, como la que se está explicando
- ▶ Por tus clases de secundaria recordarás que:

$$(a)^c(b)^c = (ab)^c$$
- ▶ Lo mismo es verdadero para exponenciación modular:

$$(m_1)^e(m_2)^e \pmod{n} = (m_1 m_2)^e \pmod{n}$$

Ataque de texto cifrado escogido

- ▶ Recordarás que:
$$c \equiv m^e \pmod{n}$$
- ▶ Con algunos despejes matemáticos se puede llegar a:
$$c_1 c_2 \pmod{n} = m_1 m_2 \pmod{n}$$

Ataque de texto cifrado escogido

Ejemplo con el código que hemos realizado:

```
1 n = public_key.public_numbers().n
2 a = 5
3 b = 10
4 encrypted_a = simple_rsa_encrypt(a, public_key)
5 encrypted_b = simple_rsa_encrypt(b, public_key)
6 encrypted_product = (encrypted_a * encrypted_b) % n
7 product = simple_rsa_decrypt(encrypted_product, private_key)
8 # product = 50 mod n!!!
```


Ataque de texto cifrado escogido

- ▶ Supón que el atacante tiene un texto cifrado c que interceptó, obtenido al cifrar con llave pública el mensaje m
- ▶ Sin la llave privada el atacante no puede descifrar el mensaje
- ▶ La víctima con la llave privada probablemente no le vaya a dar directamente el texto m descifrado
- ▶ Pero, como se explicó antes, con ingeniería social, es posible que si le de un múltiplo de m
- ▶ Con ese múltiplo, el atacante puede recuperar el original

Ataque de texto cifrado escogido

- ▶ Imagina que el atacante escogió como múltiplo el 2
- ▶ El atacante cifra con RSA y la llave pública el 2 para obtener c_r
- ▶ El texto cifrado interceptado por el atacante se denota mediante c_0
- ▶ Si se multiplica $c_r y c_0 \pmod{n}$ se obtiene c_1

Ataque de texto cifrado escogido

- ▶ $c_1 = c_0 c_r \pmod{n}$
- ▶ $c_1 = m^e r^e \pmod{n}$
- ▶ $c_1 = (mr)^e \pmod{n}$

Ataque de texto cifrado escogido

- ▶ El atacante envía c_1 a la víctima con la llave privada
- ▶ La víctima descifra el mensaje y obtiene mr , nota que no es legible y lo desecha
- ▶ El atacante logra recuperar mr
- ▶ Mediante mr el atacante quiere recuperar m
- ▶ Sin aritmética modular lo tradicional sería: $m = mr/r$
- ▶ Con aritmética modular el equivalente es:

$$m = mr * r^{-1 \bmod n} \pmod{n}$$
- ▶ $m = (mr * \text{gmpy2.powmod}(r, -1, n)) \% n$

Ejercicio

- ▶ Se reproducirá el ataque de texto cifrado escogido para cualquier mensaje

Ataque de módulo común

- ▶ Este ataque se deriva también de no poner padding en RSA
- ▶ Recuerda que el parámetro n es el módulo y es el mismo en la llave privada y en la llave pública
- ▶ Por razones matemáticas que se salen del curso, si el mismo mensaje es cifrados con llaves públicas diferentes pero comparten el mismo módulo n , entonces es posible descifrar el mensaje sin conocer la llave privada

Ataque de módulo común

- ▶ Así mismo el exponente e con el que se crean las llaves debe ser diferente
- ▶ Pero, ¿Qué tan práctico es este ataque?
- ▶ Si se usa la biblioteca cryptography vista, la probabilidad de que dos llaves públicas tengan el mismo módulo es extremadamente baja
- ▶ Tendría que haber un programador descuidado que creara manualmente las llaves y siempre le pusiera el mismo módulo
- ▶ Además el emisor tendría que haber producido el mismo mensaje y cifrarlo con dos llaves públicas diferentes

Ataque de módulo común

- ▶ Aun así recuerda que en criptografía hay que asumir que el atacante tiene medios para explotar defectos que existan
- ▶ Por ejemplo: imagina que el atacante ya estuvo recopilando varios mensajes entre un emisor y un receptor y tiene la llave pública (porque es pública)
- ▶ Luego el atacante engaña al receptor, haciéndose pasar por el emisor y le dice que ahora los mensajes se cifrarán con una nueva llave pública
- ▶ El atacante puede generar la llave con el mismo módulo que la otra pues se puede extraer el parámetro n de la llave pública
- ▶ Luego el atacante, conociendo el tipo de información que se suele intercambiar le pide al emisor que mande mensajes
- ▶ El atacante podrá juntar los mensajes que recolectó antes y después del ataque, extrayendo información y patrones de comunicación mediante el ataque de modulo común

Ataque de módulo común

- ▶ Ahora bien, en el ejemplo anterior puede notarse que el atacante, al haber engañado al receptor podrá descifrar todos sus mensajes, pues tiene control de la llave privada
- ▶ No podrá descifrar otros mensajes que no haya visto a partir de que desplegó el ataque
- ▶ Pero si podrá encontrar patrones de comunicación y determinar algunos de los mensajes que se mandaron antes de iniciar el ataque

Padding RSA

- ▶ RSA tradicional tiene dos problemas graves:
 - ▶ La salida es determinista (esto ayuda a al ataque de módulo común por ejemplo y a poder descubrir mensajes cortos por fuerza bruta)
 - ▶ Es maleable (esto ayuda al ataque de texto cifrado escogido)
- ▶ El padding ayuda a mitigar o eliminar esos problemas a partir de elementos aleatorios

Padding RSA

- ▶ El padding se aplica al texto plano antes del cifrado
- ▶ El padding asegura que los mensajes no sean demasiado pequeños
- ▶ Permite agregar suficiente estructura como para reducir la maleabilidad
- ▶ La idea es parecida a la de un IV en cifrado simétrico: el padding debe ser aleatorio y diferente cada vez que se cifra un mensaje (incluso si es el mismo) para producir mensajes cifrados que no siguen patrones

Padding RSA

- ▶ Bajo ninguna circunstancia debe usarse RSA sin padding
- ▶ Aunque la biblioteca cryptography no te deje hacerlo, otras bibliotecas como OpenSSL si te dejan
- ▶ Actualmente existen dos esquemas de padding que son usualmente utilizados:
 - ▶ PKCS #1 v1.5 (es el esquema más antiguo y debe evitarse pues tiene vulnerabilidades)
 - ▶ OAEP (Optimal Asymmetric Encryption Padding, este es el bueno)

Padding RSA

OAEP cifrar

```
1 from cryptography.hazmat.backends import default_backend
2 from cryptography.hazmat.primitives.asymmetric import rsa
3 from cryptography.hazmat.primitives import serialization
4 from cryptography.hazmat.primitives import hashes
5 from cryptography.hazmat.primitives.asymmetric import padding
6
7 message = b'test'
8 ciphertext1 = public_key.encrypt(
9     message,
10    padding.OAEP(
11        mgf = padding.MGF1(algorithm = hashes.SHA256()),
12        algorithm = hashes.SHA256(),
13        label = None)) # se usa rara vez dejar 'None'
```

Padding RSA

OAEP descifrar

```
1 recovered1 = private_key.decrypt(  
2     ciphertext1,  
3     padding.OAEP(  
4         mgf=padding.MGF1(algorithm=hashes.SHA256()),  
5         algorithm=hashes.SHA256(),  
6         label=None))
```

Padding RSA

PKCS #1 v1.5 (no usar NUNCA)

```
1 #cifrar
2 ciphertext2 = public_key.encrypt(
3     message,
4     padding.PKCS1v15())
5
6 #descifrar
7 recovered2 = private_key.decrypt(
8     ciphertext2,
9     padding.PKCS1v15())
```

Ejercicio

- ▶ Adapta el script de cifrado/descifrado con RSA para utilizar padding OAEP
- ▶ Tendrás que quitar todos los cálculos manuales con gmpy2 y usar las versiones de encrypt y decrypt de los objetos llave
- ▶ Notarás que la salida siempre es diferente sin importar que cifres el mismo mensaje

Padding RSA

- ▶ El padding también resuelve el problema de tener ceros al inicio de un mensaje
- ▶ El padding asegura que la entrada sea siempre del tamaño del módulo (usualmente 2048 bits, osea 256 bytes)
- ▶ Dado que el tamaño es conocido, esto permite que el mensaje empiece con ceros

Padding RSA

- ▶ El padding ayuda a resolver problemas de seguridad, pero no todos
- ▶ El ataque de mitm no lo puede resolver el padding, o cualquier otro problema referente a la autenticación de los mensajes
- ▶ Todavía es posible interceptar y cambiar la llave pública, el que recibe los mensajes sigue sin saber si son de la persona que piensa
- ▶ Estos son problemas que se resuelven con técnicas de autenticación (se verán en el siguiente tema del curso)

Padding RSA

- ▶ Una vez más, nunca debes usar PKCS #1 v1.5, tiene vulnerabilidades graves
- ▶ OAEP requiere de un algoritmo de hash, se recomienda SHA-256 por saberse que sigue siendo seguro, a diferencia de SHA-1 o MD5

PKCS #1 v1.5

- ▶ Este padding es vulnerable a un ataque de padding oráculo, como el que se habló puede tener AES CBC con un padding inseguro
- ▶ Este ataque es real y se sigue utilizando en servidores TLS mal configurados
- ▶ El ataque fue creado por Daniel Bleichenbacher, en su artículo: Chosen Ciphertext Attacks Against Protocols Based on the RSA Encryption Standard PKCS #1

PKCS #1 v1.5

- ▶ En este tipo de ataques el sistema básicamente le dice a cualquiera que lo use si un mensaje que se intenta descifrar tiene padding correcto (sólo el padding)
- ▶ En esencia, el ataque consiste en ir descubriendo poco a poco el contenido de un mensaje cifrado, sin conocer la llave privada, a partir de texto cifrado escogido, vulnerabilidades en el padding y la ayuda del oráculo para saber si el texto escogido da resultado

Ejercicio

- ▶ Lee el overview y las conclusiones del artículo "Chosen Ciphertext Attacks Against Protocols Based on the RSA Encryption Standard PKCS" y contesta las siguientes preguntas
- ▶ De acuerdo con el autor, ¿En qué consiste el ataque?
- ▶ ¿Qué sistemas son vulnerables?
- ▶ ¿Cómo se puede evitar el ataque?
- ▶ ¿Qué crees que tengan que ver las recomendaciones de las conclusiones con OAEP?

Más sobre RSA

Uso práctico de RSA

- ▶ Al igual que la mayoría de cifradores asimétricos, casi nunca es usado para cifrar los datos de una transmisión
- ▶ Usualmente se usa para cifrar las claves de sesión simétricas que serán usadas para cifrar los datos de transmisión
- ▶ También se utiliza en firmas digitales (más sobre eso en el siguiente tema)
- ▶ A pesar de sus limitaciones y vulnerabilidades, RSA sigue siendo ampliamente usado (aunque en ocasiones de forma incorrecta)

Manejo de llaves

- ▶ Mucha de la seguridad de un cifrador asimétrico tiene que ver con el manejo y creación de las llaves
- ▶ Al crear una llave RSA, asegúrate de hacerlo utilizando una biblioteca estable y probada, no lo hagas tu mismo de forma manual
- ▶ Mantente alerta sobre posibles bugs de seguridad en las bibliotecas que utilizas, por ejemplo que no sean suficientemente aleatorias

Manejo de llaves

- ▶ También debes tener cuidado con los parámetros de creación de llaves
- ▶ El tamaño de la llave debe ser de al menos 2048 bits, de lo contrario el módulo de la llave se puede factorizar (se pueden encontrar sus factores primos y así derivar las llaves)
- ▶ El valor del exponente e debe ser siempre 65537
- ▶ Debes asegurarte de proteger correctamente las llaves privadas, con los permisos apropiados mínimos
- ▶ Lo mejor es almacenar las llaves offline de ser posible

Manejo de llaves

- ▶ Debes considerar también almacenar la llave privada de forma cifrada, aunque esto agregue problemas en un sistema automatizado
- ▶ Lo anterior mitiga problemas si el atacante logra tener acceso al sistema de archivos

Manejo de llaves

- ▶ Ten especialmente cuidado de que la biblioteca que usas no almacena los valores p y q que son usados para generar el módulo n (son sus factores primos)
- ▶ El módulo n no es secreto (y es computacional mente intratable encontrar sus factores primos), pero p y q son los factores primos de n y si los tienes puedes derivar las llaves privadas y públicas
- ▶ El anterior no es un problema en la biblioteca cryptography (no almacena ni p ni q , ni lo necesita porque todo se calcula con e , d y n), pero otras bibliotecas si lo hacen

Manejo de llaves

- ▶ Una debilidad de la criptográfica asimétrica es que no es posible revocar llaves privadas
- ▶ Si una llave privada se compromete, los que tienen la llave pública correspondiente no tienen forma de saberlo
- ▶ En la práctica esto se resuelve con certificados (los cuales se usan en conjunto con las llave privada asimétrica), los cuales si pueden ser revocados
- ▶ Los certificados cuentan con una jerarquía de llaves y certificados, lo que hace posible hacer revocaciones
- ▶ También los certificados integran fecha de expiración lo cual ayuda a mitigar problemas de llaves comprometidas, aunque nadie se haya dado cuenta

Cómputo cuántico y RSA

- ▶ Con la llegada del cómputo cuántico en las próximas décadas, RSA será totalmente obsoleto
- ▶ El cómputo cuántico permitirá paralelizar enormemente procesos computacionales, haciendo viable la factorización de módulos grandes (como 2048 bits)
- ▶ Aunque ahora mismo no esté totalmente roto, ya debe irse reemplazando en su totalidad por otras técnicas
- ▶ De hecho el cómputo cuántico hará que muchas técnicas criptográficas modernas queden obsoletas

Breve conclusión

- ▶ Si hay algo que debes recordar es lo siguiente: los parámetros importan, las implementaciones correctas tienen detalles sutiles y evolucionan a través del tiempo
- ▶ La intuición de como funciona el cifrado asimétrico y como puede usarse es simple, pero hay muchos detalles que pueden hacer que una implementación sea segura y otra muy vulnerable
- ▶ Escoge la herramienta correcta para el trabajo correcto y escoge los parámetros correctos